

Block Round

프로젝트의 수학

Minecraft 블록 텍스처로 렌더링되는 원, 타원, 구, 타원체 생성기의 수학적 기초에 대한 기술적·교육적 문서. 각 개념에 대해 형식 정의, 기하학적 정당화, 프로젝트 소스 코드와의 직접적 대응, 게임 내 빌드 명령 (`/fill`, `/clone`, `//sphere`) 으로의 실용적 번역을 제시한다.

다루는 분야: 해석기하 · 이산 래스터화 · 디지털 위상 · 적분학 · 선형 대수 · 삼각법 · 인벤토리 산술 · 텍스처 매핑 · 팔면체 대칭.

목차

1	개요: 문제의 수학적 본질	3
2	좌표계와 이산 복셀 격자	4
3	기본 방정식: 원과 타원	4
4	유클리드 알고리즘 (거리 검사)	5
5	Bresenham 알고리즘 (정수 중점)	6
5.1	원의 경우	7
5.2	타원의 경우 (두 영역)	7
6	임계값 알고리즘 (모서리 적용 범위)	8
7	렌더링 모드: 채움, 얇음, 두꺼움	8
7.1	채움	8
7.2	얇음 (윤곽)	8
7.3	두꺼움	9
8	이산 면적 계산	10
9	2D 에서 3D 로: 타원체 방정식	10
10	복셀화와 부피 계산	11
11	기하학적 절단: 평면과 45° 대각 절단	12
12	두꺼운 3D 모드: 슬라이스를 통한 3 차원화	13
13	3D 카메라: 구면 좌표	13
14	자동 줌: 경계 구와 FOV	14
15	음영과 텍스처 곱셈	14
16	이행: 연속 수학에서 게임 내 건설로	15
17	Minecraft 인벤토리 산술	16
17.1	단일 칸, 단일 스택, 가득 찬 팩	16
17.2	저장소: 상자, 더블 상자, 셸커 상자	16
17.3	참조 표: 표준적인 구	17

18 하이라이트 오버레이와 노출된 면 카운트	17
18.1 무엇이 노출된 면으로 간주되는가	17
18.2 경계 검출 공식	17
18.3 생존 휴리스틱으로서의 노출된 면 카운트	18
19 층별 건설 (수평 분해)	18
20 팔면체 대칭을 통한 최적화 (O_h)	19
20.1 구체적인 명령 시퀀스	19
20.2 시간 비용 비교	20
21 블록 텍스처와 시각적 구성	20
21.1 여섯 면과 UV 매핑	20
21.2 블록 가족과 톤	22
21.3 블록 혼합을 통한 그라데이션	22
22 결론	22
참고문헌	23
부록 A: 건설 참조표와 실제 예제	26

1. 개요: 문제의 수학적 본질

Block Round 는 *Minecraft* 블록 텍스처로 렌더링되는 둥근 형상의 생성기로, 2D 에서는 원과 타원, 3D 에서는 구와 타원체를 다룬다. 중심 문제는 이산 래스터화 범주에 속한다: 실수 $\mathbb{R}$ 위에서 해석적으로 정의된 곡선이나 곡면이 주어졌을 때, 그것을 표현하기 위해 규칙 격자 (2D 의 픽셀, 3D 의 복셀) 의 어떤 셀을 표시해야 하는지 결정하는 문제이다. Block Round 에서는 표시된 각 셀이 *Minecraft* 팔레트의 6 면 텍스처 (잔디 블록, 조약돌, 참나무 판자, 석영, 다이아몬드 등) 를 추가로 받기 때문에, 결과 도형은 단순한 수학적 실루엣이 아니라 사용자가 생존 모드에서 재현할 수 있고, 크리에이티브 모드에서 `/fill`로 구성할 수 있으며, Sponge Schematic v2(.schem) 형식으로 WorldEdit, Litematica, MCEdit 에 가져올 수 있는 건설 가능한 구조물이 된다.

이 문제는 유일한 해를 갖지 않는다. 서로 다른 포함 기준은 서로 다른 이산 실루엣을 만들고, 각 기준은 고유한 수학적 정당성을 갖는다. 따라서 프로젝트는 세 가지 다른 2D 알고리즘 (유클리드, Bresenham, 임계값), 세 가지 렌더링 모드 (채움, 얇음, 두꺼움), 세 가지 3D 음영 스타일을 구현한다. 알고리즘 선택은 시각적 부드러움뿐만 아니라 사용자가 모아야 할 블록의 수도도 결정된다—지름 $D = 20$ 의 구는 유클리드 알고리즘에서 4189 개의 블록을 요구하지만 임계값에서는 4322 개까지 늘어난다.

실행은 전적으로 브라우저에서 이루어지며 서버 구성 요소가 없기 때문에 수학적 정식화에 추가적인 계산 효율성 요건이 부과된다 ($D = 32$ 의 다이아몬드 구는 17156 개의 텍스처 복셀을 포함해 60 fps 로 렌더링되어야 한다). 관련된 이론 영역은 다음과 같다:

- 원뿔곡선과 이차곡면의 해석기하;
- 정수 산술을 사용한 증분 알고리즘 (Bresenham);
- 디지털 위상 (4-, 6-, 26-연결 근방);
- 적분학과 계수 측도;
- 선형대수 (절단 평면, 원근 투영);
- 삼각법과 구면 좌표;
- 조합론적 인벤토리 산술과 군 대칭.

자매 프로젝트 Pixel Round 과 비교한 새로운 점. Block Round 는 렌더링되는 셀이 *Minecraft* 블록이 될 때만 발생하는 다섯 가지 새로운 수학적 관심사를 추가한다: (i) 각 복셀의 6 면 UV 매핑; (ii) 생존 모드 인벤토리 산술 (64 개 한 스택, 27 칸 단일 상자, 54 칸 더블 상자); (iii) 건설 중 노출된 면을 세는 데 사용하는 하이 라이트 오버레이; (iv) 실제 건설이 층별로 이루어지므로 3D 형상을 수평 층으로

분해; (v) /clone을 통해 수동 블록 배치 비용을 8 배 줄이기 위한 팔면체 대칭군 O_h 의 활용.

2. 좌표계와 이산 복셀 격자

캔버스는 2D 에서 $G_x \times G_y$ 셀의 격자이며, 3D 에서는 $G_x \times G_y \times G_z$ 복셀로 확장된다. 각 셀 (i, j) 는 단위 정사각형 $[i, i+1] \times [j, j+1]$ 를 차지하며, 3D 에서는 각 복셀이 단위 정육면체를 차지한다. 연속 코드에서 셀의 **중심**은 $(i + \frac{1}{2}, j + \frac{1}{2})$ 에 있다. 이 약속은 매우 중요하다: 원주를 모서리 (i, j) 와 비교하거나 중심과 비교하는 것에 따라 어떤 셀이 형상에 속하는지가 바뀌고, 따라서 사용자가 어떤 블록을 배치해야 하는지가 달라진다.

$$\text{셀 } (i, j) \text{ 의 중심} = (i + \frac{1}{2}, j + \frac{1}{2})$$

지름 D 의 형상에 대해, 코드는 격자를 각 축에서 $D+2$ 셀로 확장한다 (각 면에 한 셀 여유). 이렇게 함으로써 극단 픽셀 (북, 남, 동, 서) 이 행렬 경계에 떨어지지 않게 한다. 형상의 **중심**은 $(G_x/2, G_y/2)$ 에 있고, 반축은 $r_x = D/2$ 및 $r_y = D/2$ 이다. 3D 에서는 같은 논리가 z 축으로 확장된다. 캔버스의 단위 셀은 Minecraft 의 단위 블록과 동일하므로 알고리즘 좌표는 게임 내 (x, y, z) 좌표에 일대일로 매핑된다.

Minecraft 매핑. 플레이어가 $(0, 64, 0)$ 에서 스폰하고 지면에 놓인 $D = 16$ ($r = 8$) 구를 원한다면, 월드 좌표에서 중심은 $(0, 72, 0)$ (지표면에서 8 블록 위) 에 놓아야 구의 남극이 바닥에 정확히 닿는다. 해당 WorldEdit 명령: $(0, 72, 0)$ 에 서서 `//sphere stone 8` 실행.

3. 기본 방정식: 원과 타원

프로젝트의 모든 이론은 원점에 중심을 두고 반축이 r_x 와 r_y 인 타원의 음함수 방정식에서 시작한다:

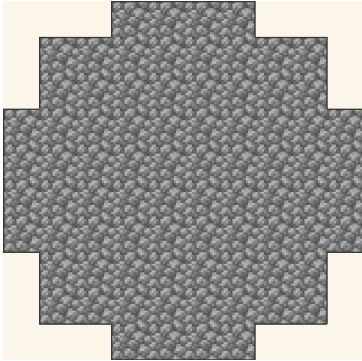
$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 = 1$$

$r_x = r_y = r$ 일 때 타원은 반지름 r 의 **원주**로 퇴화한다. 내부의 점은 ≤ 1 를 만족하고; 외부의 점은 > 1 을 만족한다. 이것이 복셀이 형상에 속하는지, 따라서 사용자가 그곳에 블록을 놓을지를 결정하는 부등식이다.

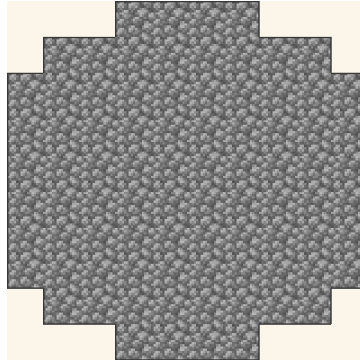
중심을 (c_x, c_y) 로 이동하고 각 셀의 중심에서 평가하면:

$$d_x = \frac{i + \frac{1}{2} - c_x}{r_x}, \quad d_y = \frac{j + \frac{1}{2} - c_y}{r_y}, \quad \text{내부} \iff d_x^2 + d_y^2 \leq 1$$

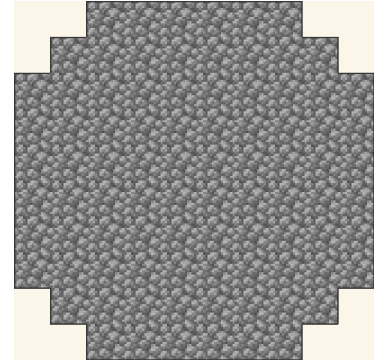
Minecraft 매핑. 생존 모드에서 탑을 짓는 경우, 사용자는 일반적으로 넓고 평평한 원을 기초로 하고 탑이 올라갈수록 점차 작은 원을 원한다. 각 층은 같은 반축과 다른 z 에서 음함수 방정식의 적용이다. Block Round 는 2D 층을 한 번만 계산하고 사용자는 `/clone`으로 층마다 복제한다.



(a) 유클리드



(b) Bresenham



(c) 임계

Figure 1: 세 가지 기준에서의 직경 $D = 10$ 인 동일한 원주. 각 그림은 대응되는 부등식 $d_x^2 + d_y^2 \leq 1$ 에 의해 표시되는 셀 집합을 보여주며, Minecraft 의 *cobblestone* 텍스처로 렌더링되어 있다. 행 사이의 전이, 모서리 셀, 기본 방위가 어떻게 달라지는지 주목하라 — 서로 다른 기준 아래에서 수학적으로 모두 옳은 세 가지 답이다.

4. 유클리드 알고리즘 (거리 검사)

아이디어. 각 행 j 에 대해, 타원 내부에 있는 열 i 를 해석적으로 찾는다. y 가 주어졌을 때 x 에 대해 풀면:

$$x = c_x \pm r_x \sqrt{1 - \left(\frac{y - c_y}{r_y}\right)^2}$$

수직 변위 $d_y = j + \frac{1}{2} - c_y$ 인 행 j 에 대해, 그 높이에서 타원의 수평 반너비는:

$$dxM = r_x \sqrt{1 - \frac{d_y^2}{r_y^2}}$$

$d_y^2/r_y^2 \geq 1$ 이면 행은 타원과 교차하지 않으므로 건너뛴다. 그렇지 않으면 $c_x - dxM \leq i + \frac{1}{2} \leq c_x + dxM$ 인 모든 열 i 를 채운다. 정수 경계는 다음에서 나온다:

$$lo = \lceil c_x - dxM - \frac{1}{2} \rceil, \quad hi = \lfloor c_x + dxM - \frac{1}{2} \rfloor$$

정당화. 항 $+ \frac{1}{2}$ 는 인덱스 i 인 셀의 중심이 $i + \frac{1}{2}$ 라는 약속에서 나온다. 함수 $\lceil \cdot \rceil$ 와 $\lfloor \cdot \rfloor$ 는 연속 구간을 정수 인덱스로 변환하여 **중심**이 타원 내부에 속하는 셀만 포함하도록 보장한다. 이 정식화는 연속 곡선에 가장 가까운 이산 근사를 만들고, 그에 따라 가장 부드러운 시각적 실루엣을 만든다. 그러나 작은 지름에서는 결과가 다른 알고리즘에 비해 픽셀 아트 특성이 덜 두드러질 수 있다— $D = 5$ 의 유클리드 원은 13 개의 블록만 갖는 반면, 같은 지름의 임계값 원은 21 개를 갖는다.

Minecraft 맵핑. 유클리드 기준은 WorldEdit 의 `//sphere`가 만드는 것에 가장 가깝다. 유클리드 셀만 사용해 $D = 16$ ($r = 8$) 조약돌 구를 만들 때, 부피는 채움 모드에서 $V \approx 2145$ 블록, 얇은 모드 (셀) 에서 약 804 블록이 된다—34 팩 ($34 \times 64 = 2176$) 을 모으는 것과 13 팩 ($13 \times 64 = 832$) 을 모으는 것의 차이이다.

5. Bresenham 알고리즘 (정수 중점)

Bresenham 알고리즘은 1965 년 선분 그리기를 위해 발표되었고, 후에 원으로 확장되었으며 Pitteway 와 Kappel 에 의해 임의의 타원으로 일반화되었다. 그 특징적인 성질은 부동소수점 연산과 제곱근 추출을 사용하지 않는 것이다: 다음 픽셀의 결정은 두 후보 픽셀 사이의 중점이 해석 곡선의 어느 쪽에 있는지를 평가하는 결정 함수를 통해 **정수 덧셈과 비교**만 사용한다. Minecraft 건축에서는 이것이 가장 인식 가능한 픽셀 아트 외관을 주는 알고리즘이다—커뮤니티가 2011 년 부터 손으로 지어온 것과 같은 계단형 실루엣이다.

5.1 원의 경우

정수 반지름 R 의 원에 대해, 알고리즘은 $(x=0, y=R)$ 에서 초기 결정 매개변수와 함께 시작한다:

$$d_0 = 1 - R$$

각 단계에서 (0에서 45° 의 8분원 내에서):

$$\begin{cases} \text{만약 } d < 0 : & \text{다음은 } (x+1, y), \quad d += 2x + 3 \\ \text{만약 } d \geq 0 : & \text{다음은 } (x+1, y-1), \quad d += 2(x - y) + 5 \end{cases}$$

정당화. 함수 d 는 각 반복에서 두 후보 픽셀 옵션 사이의 **중점**에서 평가된 원의 음함수 방정식 값을 근사한다:

$$F(x+1, y-\frac{1}{2}) = (x+1)^2 + (y-\frac{1}{2})^2 - R^2$$

d 의 부호는 중점이 원 내부에 있는지 (수평으로만 전진) 외부에 있는지 (수직 좌표도 감소) 나타낸다. 원의 8개 8분원은 이면체군 D_4 의 대칭성을 계산된 8분원에 적용하여 얻으며, 이는 코드에서 8번의 `span(...)` 호출에 해당한다. 결과 궤적은 부드러운 곡선의 이산 근사에 특징적인 계단형 패턴을 보인다.

5.2 타원의 경우 (두 영역)

반축 a, b 의 타원에 대해, 알고리즘은 접선이 45° 를 이루는 점을 경계로 제 1 사분면을 **두 영역**으로 나눈다:

$$\text{영역 I: } 2b^2x < 2a^2y \quad (|dy/dx| < 1), \quad \text{영역 II: 그렇지 않으면}$$

초기 결정 매개변수는:

$$\begin{aligned} p_1 &= b^2 - a^2b + \frac{a^2}{4}, \\ p_2 &= b^2\left(x + \frac{1}{2}\right)^2 + a^2(y-1)^2 - a^2b^2. \end{aligned}$$

각 반복에서 p 는 사전 계산된 증분 (4를 곱한 후 모두 정수)으로 업데이트된다. 결과는 픽셀당 최소 산술: 한 번의 부호 검사와 두 번의 덧셈.

Minecraft 맵핑. 생존 빌더에게 Bresenham 실루엣은 이산 단계를 손으로 세기 쉽기 때문에 유클리드보다 선호되는 경우가 많다—사용자는 다음 블록이 항상 직선이거나 대각선임을 알 수 있어, 배치 시 손 위치가 일관된다. $D=11$ 의 Bresenham 원은 수십 개의 커뮤니티 가이드에 게재된 표준 80 블록 시퀀스를 갖는다.

6. 임계값 알고리즘 (모서리 적용 범위)

이 알고리즘은 묻는다: 셀의 어떤 모서리가 타원 내부에 있는가? 각 셀 (i, j) 에 대해 코드는 형상의 중심에 가장 가까운 모서리 (셀 가장자리에 대한 중심 투영) 를 찾는다:

$$n_x = \text{clamp}(c_x, i, i+1), \quad n_y = \text{clamp}(c_y, j, j+1)$$

$$\text{내부} \iff \left(\frac{n_x - c_x}{r_x} \right)^2 + \left(\frac{n_y - c_y}{r_y} \right)^2 \leq 0,94$$

값 0,94는 1,0 보다 약간 낮은 경험적 임계값이다. 그 목적은 곡선이 셀의 전체 가장자리에 접하는 네 기본 방위 (북, 남, 동, 서) 에서 발생하는 1 픽셀 접선 스파이크 아티팩트를 제거하는 것이다.

세 알고리즘 중 동일한 지름 D 에서 최대 면적 실루엣을 만들어내는 것이 이것이다. 포함 조건이 가장 덜 제한적이기 때문이다. Minecraft 에서는 더 많은 블록을 사용하는 대가로 가장 둥근 외관을 제공하는 알고리즘이다— $D = 16$ 은 임계값에서 213 개의 셀 블록, Bresenham 에서 200 개, 유클리드에서 192 개를 만든다.

Minecraft 맵핑. 임계값은 건물이 멀리서 보일 때 (예: 사원 위의 석영 돔) 선택지인데, 약간 더 큰 실루엣이 환경 차폐 그림자를 통해 더 명확하게 읽히기 때문이다. 트레이드오프: 유클리드에 비해 셀당 약 5–10% 더 많은 블록.

7. 렌더링 모드: 채움, 얇음, 두꺼움

내부 셀에 대해 $f[j][i] = 1$ 인 행렬을 가지면, 프로젝트는 **이산 위상**을 통해 세 가지 시각적 스타일을 도출한다:

7.1 채움

f 를 변경하지 않고 반환한다. 모든 내부 셀이 블록이 된다. 이것이 `//sphere stone 8`이 만드는 것이다.

7.2 얇음 (윤곽)

셀이 채워져 있고 적어도 하나의 직교 이웃 (북, 남, 동, 서) 이 형상의 외부에 있으면 얇은 윤곽에 속한다. 논리 표기:

$$b(i, j) = f(i, j) \wedge (\neg f(i-1, j) \vee \neg f(i+1, j) \vee \neg f(i, j-1) \vee \neg f(i, j+1))$$

기하학적으로: 이것은 형상의 **이산 경계**로, 위상 경계 $\partial F = F \cap \overline{F^c}$ 의 이산 유사물이다. Minecraft에서는 `//hsphere`(빈 구)에 해당한다: 셀만 구체화된다.

7.3 두꺼움

두꺼운 모드는 경계 픽셀에 **대각 다리**를 추가한다: 수평 경계 이웃과 수직 경계 이웃을 동시에 갖는 내부 셀. 이것은 얇은 모드가 남기는 1 픽셀 대각선을 닫아, 윤곽이 장면에서 안정적으로 보여야 할 때 (즉 45° 구멍이 없어야 할 때) 유용하다—특히 유리 와 얼음 돔에 중요하다.

$$\begin{aligned} t(i, j) &= b(i, j) \vee (f(i, j) \wedge h_H \wedge h_V), \\ h_H &= b(i-1, j) \vee b(i+1, j), \\ h_V &= b(i, j-1) \vee b(i, j+1). \end{aligned}$$

Minecraft 매핑. $D = 20$ 의 유리 돔 (비어있음)에 대해, 두꺼운 모드는 608 블록의 기밀 셀을 만들고 얇은 모드는 약 510 블록이다—추가 ~ 100 블록은 얇은 모드가 남기는 비어 있는 대각 셀 (공기 블록)을 채워, 그 공기 셀들로 환경광이 새어 들어가는 것을 막는 대각선 다리이다.

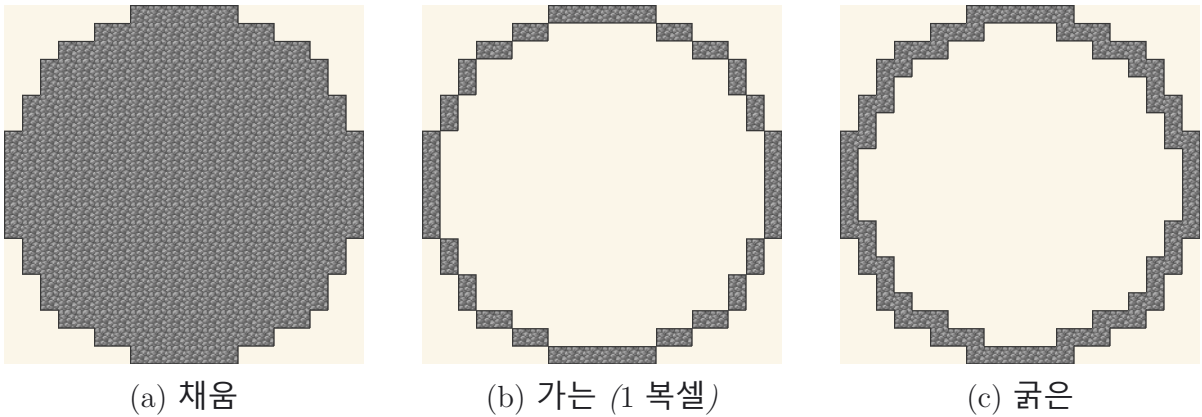


Figure 2: 동일한 $D = 20$ 유클리드 원주에 대한 세 가지 렌더링 모드. 가는 모드는 이산 위상 경계 (∂F)에 대응하고, 굵은 모드는 45° 의 구멍을 막기 위해 대각 블록을 추가하는데, 이는 밀폐 구조 (수중 유리/얼음 돔)에 필수적이다.

8. 이산 면적 계산

area2D 함수는 단순히 f 에 표시된 셀을 **센다**. 이것이 계수 측도로, 타원의 지시 함수의 리만 적분을 근사한다:

$$\pi r_x r_y \text{ 연속 면적} \quad \longleftrightarrow \quad \sum_{j=0}^{G_y-1} \sum_{i=0}^{G_x-1} f(i, j) \text{ 이산 면적}$$

큰 D 의 경우 이산 면적/연속 면적 $\rightarrow 1$. 작은 D 의 경우 알고리즘 간 차이가 보인다: 임계값은 과대평가하는 경향이 있고, 유클리드는 이상 면적에 가깝게 머물며, Bresenham 은 둘 사이 어딘가에 위치한다.

Minecraft 맵핑. Block Round UI 의 블록 수 칩은 바로 이 수를 표시한다: $D = 8$ 에서 268, $D = 12$ 에서 904, $D = 16$ 에서 2145. 칩은 수를 N (스택 $\times 64 +$ 나머지) 로 표현한다—그래서 $2145 = 33 \times 64 + 33$, 즉 “33 스택 더하기 33 개”로, 이것이 사용자가 실제로 모아야 하는 것이다.

9. 2D 에서 3D 로: 타원체 방정식

3D 에서 기본 형상은 (c_x, c_y, c_z) 에 중심을 두고 반축 r_x, r_y, r_z 를 갖는 **타원체**이다. 그 방정식은 타원의 자연스러운 일반화이다:

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$

$r_x = r_y = r_z$ 일 때 **구**로 돌아간다. 각 복셀의 중심에서 평가하면:

$$a_\xi = \frac{\xi + \frac{1}{2} - c_\xi}{r_\xi} \quad (\xi \in \{x, y, z\}), \quad \text{내부} \iff a_x^2 + a_y^2 + a_z^2 \leq 1$$

Minecraft 맵핑. $W = 20, H = 10, D = 20$ 의 타원체는 지하 돔이나 경기장 지붕에 이상적인 납작한 구를 만든다. 부피 약 2094 블록 (33 스택). 동등한 WorldEdit 명령: `//ellipsoid stone 10 5 10`.

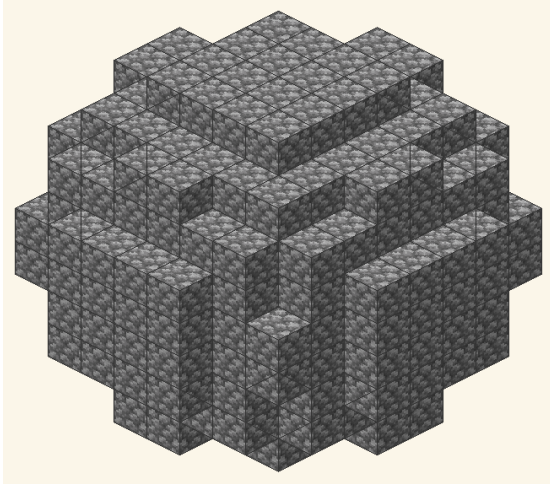
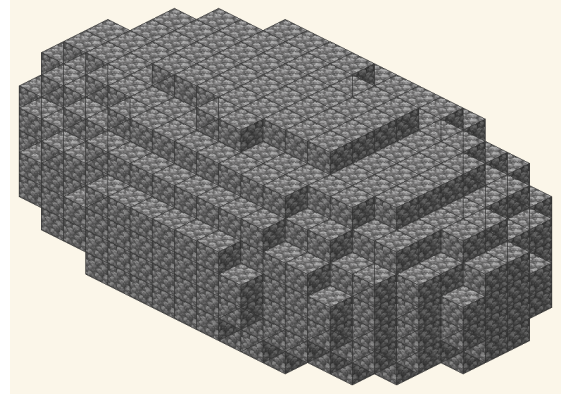
(a) 구 $D = 10$, $r_x = r_y = r_z = 5$.(b) 타원체 $W = 20$, $H = 10$, $D = 12$.

Figure 3: $\mathbb{R}^3$ 에서의 복셀화. 각 정육면체가 하나의 복셀이며, 그 중심 좌표 (i, j, k) 를 음함수 방정식 $a_x^2 + a_y^2 + a_z^2 \leq 1$ 에 대입해 판정한다. 표면에 보이는 계단 모양은 이산화의 수학적 귀결이며 — 바로 Minecraft에 시각적 정체성을 부여하는 그 특성이다.

10. 복셀화와 부피 계산

타원체의 연속 부피는 적분학의 고전적 결과로, 원판에 대한 적분으로 얻어진다:

$$V = \frac{4}{3} \pi r_x r_y r_z$$

이산 버전은 상자 $D_x \times D_y \times D_z$ 의 각 복셀을 순회하고 $a_x^2 + a_y^2 + a_z^2 \leq 1$ 일 때 카운터를 증가시킨다. 이것이 `voxelVolume` 알고리즘이다.

셀 vs. 내부. *filled* 모드에서 프로젝트는 보존 집합 외부에 적어도 하나의 6-이웃이 있는 복셀 (즉, 외부 표면이나 절단면에서 보이는 복셀)만 표시한다. *thin*에서는 완전한 타원체의 **표면**만 표시한다 (1 복셀 두께). 생존 모드 게임에서는 보이는 면만 전용 텍스처 블록이 필요하다—내부 복셀은 자원을 절약하기 위해 가장 저렴한 블록 (다이아몬드 대신 흙)으로 채울 수 있다.

참조 표. 아래 표는 Minecraft 커뮤니티 빌드에서 가장 많이 요청되는 표준 구 지름을 블록 수 (V), 64 짜리 팩 수 ($P = \lceil V/64 \rceil$), $54 \times 64 = 3456$ 슬롯의 더블 상자 수 ($DC = \lceil V/3,456 \rceil$)로 변환하여 요약한다. 건설 전에 자원 채집을 계획하는 데 사용한다.

지름	블록 (V)	팩 ($\times 64$)	더블 상자	메모
D=8	268	5	1	작은 돔
D=12	904	15	1	중간 구
D=16	2145	34	1	큰 탑 꼭대기
D=20	4189	66	2	경기장 지붕
D=24	7238	114	3	큰 돔
D=32	17156	269	5	메가 빌드

$D \in \{8, 12, 16, 20, 24, 32\}$ 에서 유클리드 알고리즘으로 계산된 채움 모드 부피. 얇은 모드 (셀) 값은 대략 $V_{\text{shell}} \approx 4\pi r^2$ 블록 (1 복셀 두께), 즉 D 에 따라 채움 부피의 25% 에서 40% 사이.

11. 기하학적 절단: 평면과 45° 대각 절단

Block Round 는 세 평면으로 솔리드를 절단할 수 있다. 각 절단은 복셀을 버리는 선형 부등식이다:

$$\begin{aligned} \text{X 축: } x &\geq cut &\Rightarrow \text{버려짐} \\ \text{Y 축: } y &\geq cut &\Rightarrow \text{버려짐} \\ \text{대각: } x + y &\geq cut &\Rightarrow \text{버려짐} \end{aligned}$$

X 와 Y 평면은 좌표축에 수직이며, 법선 벡터는 $(1, 0, 0)$ 과 $(0, 1, 0)$ 이다. 대각 평면은 법선 $(1, 1, 0)/\sqrt{2}$ 를 가지며 XY 평면에서 45° 로 자른다: 수학적으로는 평면족 $x + y = k$ 이다. $D_x \times D_y$ 상자에서 $x + y$ 의 최대값이 $D_x + D_y$ 이므로, 대각 절단 슬라이더 범위는 $D_x + D_y$ 이고 X 와 Y 는 각각 D_x 와 D_y 이다. 절단은 **비례적**이다: 코드는 백분율 $cutPct$ 을 저장하고 한계를 다음과 같이 재계산한다:

$$cut = cutPct \cdot \max_{\text{축}}$$

따라서 한 축에서의 50% 는 사용자가 축을 전환하거나 크기를 조정해도 50% 로 유지된다.

Minecraft 맵핑. 구에 대한 Y 축 50% 절단은 부피의 절반인 표준적인 돔을 만든다. $D = 16$ 의 경우: $V_{\text{full}} = 2145$ 블록, $V \approx 1095$ 블록 (약 18 팩). 50% 의 대각 절단은 분수 내부와 원형 극장 좌석에 자주 사용되는 반접시 단면을 만든다.

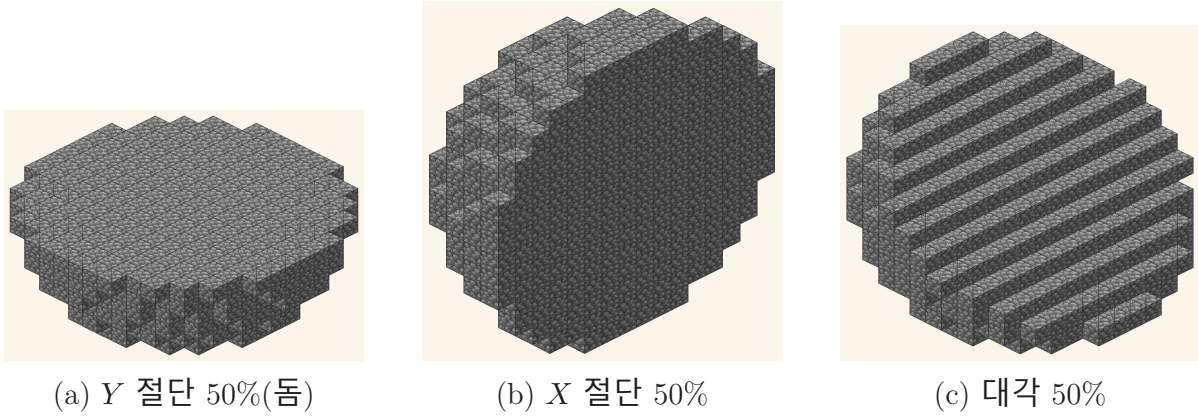


Figure 4: 동일한 $D = 16$ 구에 대한 세 가지 절단으로, 모두 부피의 50%를 보존한다. Y 절단은 고전적인 돔 $V_{\text{dome}} = \frac{2}{3}\pi r^3$ 을 만들고, 대각 절단은 그 절단면 특유의 빗변 $x + y = k$ 를 드러낸다.

12. 두꺼운 3D 모드: 슬라이스를 통한 3 차원화

3D에서 *thick* 모드의 경우, 프로젝트는 세 축을 따라 **모든 슬라이스**에 2D 두꺼운 알고리즘을 적용한다: 각 z 에 대한 XY , 각 y 에 대한 XZ , 각 x 에 대한 YZ . 그런 다음 결과의 합을 취한다. 기하학적 동기는 기밀성이다: 셀 두께를 두 배로 하지 않고 모든 대각선을 막는 최소 복셀 집합. 형식적으로, $S_z(z)$ 를 높이 z 의 XY 슬라이스로, T 를 2D 두꺼운 연산자로 하면:

$$\text{thick}3D = \bigcup_z T(S_z(z)) \cup \bigcup_y T(S_y(y)) \cup \bigcup_x T(S_x(x))$$

결과는 그 다음 절단에 의해 보존된 집합과 교차된다. 배경 이론은 **디지털 위상**이다: 두꺼운 연산자는 셀의 26-연결성 (대각선을 포함한 이웃)을 보장한다. 이는 대각으로 인접한 복셀이 면으로 닿지 않는 Minecraft 건설에 유용하다—빛, 물, 생명체가 벽이 없는 것처럼 대각 간격을 통과한다.

Minecraft 매핑. 생존 모드의 바다 사원 위 수중 유리 돔에는 두꺼운 모드가 필수이다: 대각 다리는 얇은 모드가 남기는 비어 있는 대각 셀 (공기 블록)을 채워, 이 공기 셀들을 통해 물이 새어 들어오는 것을 막는다. 비용: $D = 20$ 돔은 두꺼운 모드에서 ≈ 380 개의 유리 블록을, 얇은 모드에서 ≈ 320 개를 필요로 한다—완전 밀폐를 위한 19% 더 많은 재료.

13. 3D 카메라: 구면 좌표

3D 카메라는 두 각도와 함께 거리 r 로 객체 주위를 공전한다— θ (방위각, 수평면)와 φ (극각, 수직축에서). 이것은 구면 좌표의 **물리적 관례**이며, 직교 좌표 (*three.js*

가 기대하는 형식) 로의 변환은:

$$\begin{aligned}x &= r \sin(\varphi) \cos(\theta), \\y &= r \cos(\varphi), \\z &= r \sin(\varphi) \sin(\theta).\end{aligned}$$

초기 위치: $\theta = \pi/4(45^\circ, \text{ 등각 투시도})$ 와 $\varphi = \pi/3(\text{북극에서 } 60^\circ, \text{ 적도보다 약간 위})$. 마우스 드래그는 θ 와 φ 를 직접 증분한다; 휠/핀치는 r 을 증분한다. 이 매개변수화의 단순성이 사원수나 명시적 행렬 없이 자유 회전을 가능하게 한다.

14. 자동 줌: 경계 구와 FOV

사용자가 도형 크기를 변경하거나 카메라를 재설정하면 프로젝트는 객체가 어떤 각도에서도 뷰포트에 맞도록 **이상적 거리**를 다시 계산한다. 아이디어는 객체를 반지름 R 의 구 (AABB 의 대각선의 절반, 축 정렬 경계 상자) 안에 가두는 것이다:

$$R = \sqrt{(D_x/2)^2 + (D_y/2)^2 + (D_z/2)^2}$$

구는 **회전 불변 투영**을 가지므로 한 방향에서 뷰포트에 맞으면 모든 방향에서 맞는다. 카메라의 수직 FOV($fov_V = 35^\circ$ 라디안) 가 주어지면 반지름 R 의 구를 프레임에 맞추는 데 필요한 거리는:

$$dist_V = \frac{R}{\tan(fov_V/2)}$$

수평 FOV 는 캔버스 종횡비에서 다음과 같이 나온다:

$$fov_H = 2 \arctan(\tan(fov_V/2) \cdot aspect)$$

최종 거리는 $dist_V$ 와 $dist_H$ 의 최대값에 1.20(20% 시각 여유) 를 곱한 것이다. 도형이 절단되면 코드는 D_x 또는 D_y 를 남은 크기로 축소한다. 참나무 또는 크리퍼 이스터에그가 활성화되면 경계 상자가 엔티티 높이를 포함하도록 위로 확장된다.

15. 음영과 텍스처 곱셈

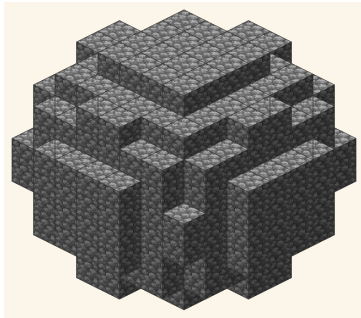
세 가지 3D 스타일 (*Classic*, *Smooth*, *Blocks*) 은 각 복셀의 세 가지 면 (위, 밝은 면, 어두운 면) 에 적용되는 **곱셈 인자**로 구분된다. 함수 `shadeHex(hex, factor)` 는 `hex` 를 (R, G, B) 로 파싱하고 각 채널을 $[0, 255]$ 로 클램프하면서 곱한다—결과는 블록의 원시 텍스처 위에 적용되는 색조가 된다:

$$R' = \text{clamp}(R \cdot f), \quad G' = \text{clamp}(G \cdot f), \quad B' = \text{clamp}(B \cdot f)$$

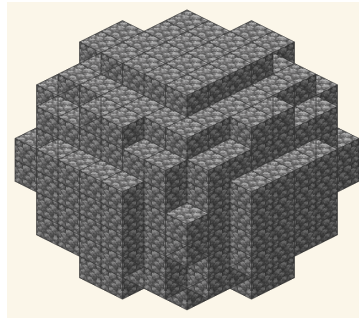
이것은 이상적인 램베르트 조명 함수의 선형 근사이다,

$$I = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot \text{emissive_factor} \cdot \text{texture}$$

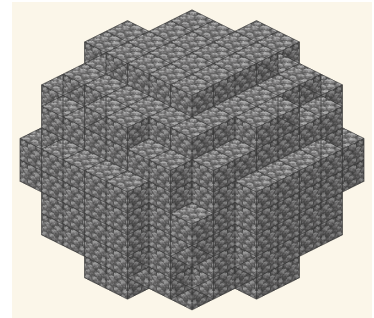
세 인자는 각 면의 법선과 고정된 광원 방향 사이 각도의 코사인에 해당한다. *Smooth*에서는 인자가 비슷하고 (유사한 면), *Classic*에서는 인자가 더 다르다 (강한 대비, 바닐라 Minecraft의 외관). *Blocks*는 추가로 복셀에 기하학적 들여쓰기를 도입한다—인접 복셀이 동일한 텍스처를 가져도 복셀 간 경계가 드러나도록 각 면을 안쪽으로 일정 평행이동한다. 발광 블록 (글로우스톤, 바다 랜턴, 슈룸라이트, 마그마, 우는 흑요석)의 경우 램베르트 항은 상수 $I = \text{emissive_factor} \cdot \text{texture}$ 로 대체되며, 이는 블록의 MC 광 레벨에 맞게 조정된다.



(a) *Classic* (강한 대비)



(b) *Blocks* (기본)



(c) *Smooth* (면이 유사)

Figure 5: 같은 세 개의 가시면에 적용된 세 가지 빛 배율. *Classic*은 표준적인 Minecraft 바닐라 외관을 주며, *Smooth*는 매끈한 표면을 위해 대비를 낮추고, *Blocks*는 텍스처가 같아도 복셀 경계를 드러내는 기하학적 인셋이 있는 중간 기본값이다.

16. 이행: 연속 수학에서 게임 내 건설로

위의 15개 절은 Block Round가 자매 프로젝트 Pixel Round와 공유하는 연속에서 이산으로의 파이프라인을 설명한다: 타원의 음함수 방정식에서 결과를 구도 잡는 카메라까지. 나머지 6개 절은 Block Round에 **특유한** 것을 설명한다—렌더링되는 셀이 추상적 픽셀이 아니라 *Minecraft* 블록이라는 사실로 인해 도입되는 추가 수학 층, 각각 6개의 텍스처 면, 인벤토리에서의 무게, 채집하고 배치하는데 필요한 플레이 시간 비용을 가진다.

이는 장식적 고려가 아니다. 블록 텍스처로 렌더링하는 결정은 사용자가 직면하는 운영적 문제를 바꾼다: *PNG*를 내보내는 대신 사용자는 실루엣을 배치 계획으로 번역해야 한다. 다음 절의 수학적 내용은 정확히 이 번역에 관한 것이다: 인벤토리 산술, 면 수 최적화, 층별 분해, 그리고 N 개의 블록 배치 비용을 $N/8 + 7$ 번의 명령 호출로 줄이기 위한 대칭의 활용.

17. Minecraft 인벤토리 산술

Minecraft 플레이어는 9×4 격자로 구성된 36 칸 인벤토리에 아이템을 휴대한다. 각 칸은 같은 종류의 아이템을 최대 64 개까지 보유한다 (한 스택 또는 팩). 플레이어는 저장소도 사용할 수 있다: 단일 상자 27 칸, 더블 상자 54 칸. 목표 블록 수를 채집 계획으로 변환하는 산술은 따라서 올림 나눗셈 문제이다.

17.1 단일 칸, 단일 스택, 가득 찬 팩

스택은 인벤토리 회계의 기본 단위이다. 각 스택은 최대 64 개의 아이템을 포함한다. 블록 수 N 에서 팩 (또는 스택) 으로의 변환은

17.2 저장소: 상자, 더블 상자, 셸커 상자

건축에 더블 상자가 몇 개 필요한지 계산하려면 블록 수를 $54 \text{ 칸} \times 64 \text{ 아이템/칸} = 3456 \text{ 아이템/더블 상자}$ 로 나눈다:

$$\text{팩 : } N_{\text{packs}} = \left\lceil \frac{N}{64} \right\rceil \quad \text{더블 상자 : } N_{\text{dc}} = \left\lceil \frac{N}{3456} \right\rceil$$

단일 상자는 $27 \times 64 = 1728$ 개의 아이템을 보관한다. 셸커 상자 (휴대용 컨테이너) 는 $27 \times 64 = 1728$ 개의 아이템을 보관하지만 그 자체를 상자 칸 안에 저장할 수 있으므로 54 개의 셸커 상자로 채워진 더블 상자는 최대 $54 \times 1728 = 93312$ 개의 아이템을 보관할 수 있다. 메가 빌드의 경우 관련 용량 단위는 셸커로 적재된 더블 상자이다.

17.3 참조 표: 표준적인 구

아래 표는 표준적인 채워진 구 부피를 채집 계획으로 변환한다. 마지막 열은 각 지름에 대한 빌드 맥락을 제안한다:

지름	V (블록)	팩	저장	전형적인 빌드
D=8	268	5	1 더블	오두막 지붕, 우물 돔
D=12	904	15	1 더블	감시탑 꼭대기
D=16	2145	34	1 더블	마을 중, 도서관 돔
D=20	4189	66	2 더블	천문대 돔
D=24	7238	114	3 더블	사원 돔
D=32	17156	269	5 더블	대성당 돔, 월드 보스 경기장

Block Round의 정보 칩이 표시하는 스택과 나머지 표현 $N = q \cdot 64 + r$ 은 생존 인벤토리가 부분 스택을 표시하는 방식을 반영한다: 플레이어는 q 개의 가득 찬 칸과 r 을 보이는 한 칸을 본다. $D = 16$ 의 경우, 칩은 “2145 ($33 \times 64 + 33$)”을 출력하여 플레이어에게 33 칸을 완전히 채우고 한 칸에 33 개의 아이템을 채우라고 알린다—정확히 34 칸이다.

18. 하이라이트 오버레이와 노출된 면 카운트

Block Round는 기본적으로 각 보이는 복셀 면의 가장자리에 검은 하이라이트 오버레이를 렌더링한다. 시각적으로는 게임 내 디버그 오버레이의 “F3+G” 체크 경계 스타일과 일치하며, 복셀 수준으로 축소된다. 수학적으로 오버레이는 **노출된 면 카운트**의 시각화를 구현한다: 생존 빌더가 블록 단위로 진행을 검증하는 데 사용하는 수.

18.1 무엇이 노출된 면으로 간주되는가

복셀 v 가 이웃 n 과 공유하는 면은 $n \notin V_{\text{solid}}$ 일 때 (이웃이 비어 있을 때) 그리고 오직 그 때에만 노출되어 있다. 각 단단한 복셀은 0(완전히 묻힘)에서 6(고립됨)까지의 면을 기여한다. 총 노출된 면 수 F_{exp} 는 도형의 표면적의 이산 유사물이다.

18.2 경계 검출 공식

복셀이 경계에 있는 것은 그 여섯 면 이웃 중 적어도 하나가 단단한 것 외부에 있는 경우이다:

$$b(i, j, k) = f(i, j, k) \wedge (\neg f(i \pm 1, j, k) \vee \neg f(i, j \pm 1, k) \vee \neg f(i, j, k \pm 1))$$

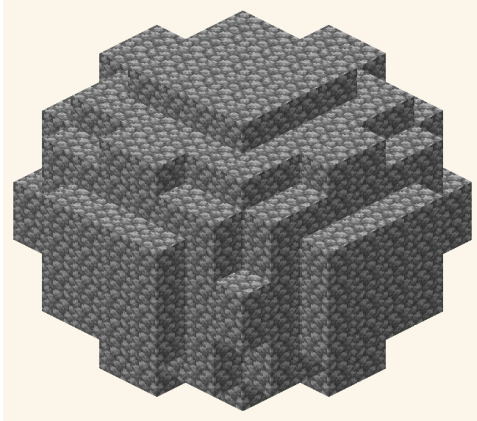
이는 3D 로 확장된 2D 얇은 알고리즘과 동일한 경계 연산자이며, 오버레이는 단순히 각 경계 복셀의 큐브 모서리 선을 그린다. 유리와 얼음의 경우 기본값은 OFF, 조약돌, 돌, 다이아몬드 및 기타 불투명 블록의 경우 기본값은 ON 이다.

18.3 생존 휴리스틱으로서의 노출된 면 카운트

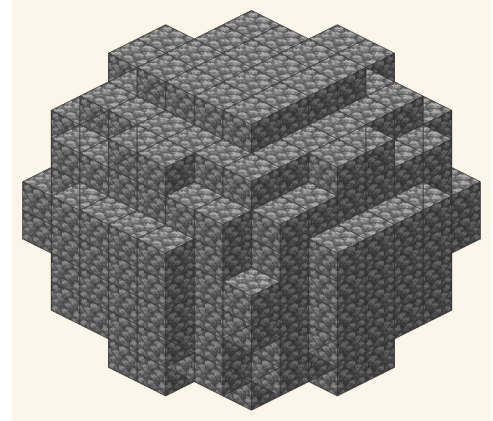
생존 빌더에게 관련된 수는 총 부피 V 가 아니라 노출된 면 수 F_{exp} 이다. 그것이 외부에서 보이기 때문이다. 모든 단단한 복셀에 대한 F_{exp} 의 합은 경계 복셀 수의 6 배에서 경계 복셀 사이 면 인접의 두 배를 뺀 것과 같다:

$$F_{\text{exp}} = \sum_{\mathbf{v} \in V_{\text{solid}}} \sum_{\mathbf{n} \in N_6(\mathbf{v})} [\mathbf{v} + \mathbf{n} \notin V_{\text{solid}}]$$

$D = 16$ 의 구 (채움 모드) 의 경우, $V = 2145$, $V \approx 432(\text{셀})$, $F_{\text{exp}} \approx 1184$ 가시 면 — 건설은 약 432 개의 가시 블록만 필요하고, 나머지 1713 블록은 내부이며 가장 저렴한 블록 (흙, 조약돌) 으로 채울 수 있다. 오버레이는 빌더가 셀의 누락된 블록을 즉시 발견할 수 있게 한다.



(a) 오버레이 OFF



(b) 오버레이 ON(기본)

Figure 6: 노출된 모서리의 검은 윤곽선 유무에 따른 동일한 복셀 집합의 비교. *highlight overlay* 가 ON 이면 겹데기에 빠진 블록이 한눈에 알아챌 수 있는 패턴 불연속을 만든다 — 서바이벌 건축 중의 빠른 검증 도구.

19. 층별 건설 (수평 분해)

Minecraft 에서 건설은 수평 층별로 진행된다: 플레이어는 한 층에 서서 위 층의 블록을 배치한 다음 한 블록 위로 올라간다 (점프 + 임시 비계 블록 배치 또는 엘리트라 사용). 따라서 3D 타원체 문제는 층마다 하나씩 2D 타원의 더미로 가장 잘 이해된다:

$$\text{층}(k) = \{ (i, j) : a_x(i)^2 + a_y(j)^2 \leq 1 - a_z(k)^2 \}$$

$-r_z$ 에서 $+r_z$ 까지의 각 정수 층 k 에 대해, 횡단면 자체는 축소된 반축 $r_x(k)$ 와 $r_y(k)$ 를 갖는 타원이다:

$$r_x(k) = r_x \sqrt{1 - \left(\frac{k}{r_z}\right)^2}, \quad r_y(k) = r_y \sqrt{1 - \left(\frac{k}{r_z}\right)^2}$$

이는 3D 문제를 이전 절의 2D 알고리즘을 한 번에 한 층씩 계산하고 결과를 쌓는 것으로 환원한다는 것을 의미한다. 인지적으로 이는 거대한 단순화이다: 세 좌표를 정신적으로 추적하는 대신 빌더는 층당 두 좌표와 층 인덱스를 추적한다.

Minecraft 맵핑. $D = 16(r_z = 8)$ 의 구의 경우, 적도 층 ($k = 0$) 은 완전한 $D = 16$ Bresenham 원이다 (약 200 블록). 다음 층 위로 ($k = 1$) 는 $D = 15,97$ 원이다. $k = 4$ 에서 층은 $D = 13,86$ 원 (약 148 블록) 이고, $k = 7$ 에서는 $D = 7,48$ 원 (약 43 블록) 으로 축소된다. 극점 ($k = \pm 8$) 은 단일 블록 캡이다.

Block Round 정보 칩은 사용자가 중앙 가이드 모서리 버튼 (키 c) 을 활성화할 때 3D 모드에서 이 층 수를 표시한다. 수평 슬라이스는 적도와 각 $r_z/2$ 세분에서 반투명 노란색 십자로 그려진다.

20. 팔면체 대칭을 통한 최적화 (O_h)

원점을 중심으로 하는 구는 차수 48 의 **팔면체군** O_h 의 작용 하에서 불변이다. 구의 복셀화는 세 좌표 반사와 각 축 주위의 세 $\pi/2$ 회전에 의해 생성된 차수 48 의 부분군을 보존한다. 실제 건설 목적에 관련된 부분군은 $x \mapsto -x, y \mapsto -y, z \mapsto -z$ 에 의해 생성된 차수 8 의 팔분원 반사군 $\mathbb{Z}_2^3$ 이다. 한 팔분원의 복셀 집합이 전체 구를 결정한다:

$$V_{\text{total}} \approx 8 V_{\text{octant}} \implies \text{감소} = \frac{N - N/8}{N} = 87,5\%$$

이것은 Minecraft 빌더가 사용할 수 있는 가장 큰 수학적 최적화이다. N 개의 블록을 수동으로 배치하는 대신, 빌더는 양의 팔분원에 $N/8$ 개의 블록을 배치하고 적절한 반사로 7 개의 `/clone`(바닐라) 또는 `//copy + //paste`(WorldEdit) 명령을 발행한다.

20.1 구체적인 명령 시퀀스

월드 좌표 $(0, 72, 0)$ 를 중심으로 하는 $D = 24(r = 12, V = 7238 \text{ 블록})$ 의 구의 경우, 먼저 양의 팔분원 ($+x, +y, +z$ 영역, 약 905 블록) 을 건설한 다음 발행한다:

```

/clone 0 72 0 12 84 12 ~~~ filtered minecraft:stone
/clone 0 72 0 12 84 12 -12 72 0 mode:masked (-x 반사)
/clone 0 72 0 12 84 12 0 60 0 mode:masked (-y 반사)
/clone 0 72 0 12 84 12 0 72 -12 mode:masked (-z 반사)
/clone -12 72 0 -1 84 12 mode:masked (-x, -y 팔분원)
/clone 0 60 -12 12 71 -1 mode:masked (-y, -z 팔분원)
/clone -12 72 -12 -1 84 -1 mode:masked (-x, -z 팔분원)
/clone -12 60 -12 -1 71 -1 mode:masked (-x, -y, -z 팔분원)

```

각 `/clone`은 약 50ms의 서버 시간을 소비하므로 7개의 반사는 0.5초 이내에 완료된다. 팔분원의 905 블록 수동 배치는 생존에서 약 15분, 브러시가 있는 크리에이티브에서 약 45초가 걸린다. 대칭 없이 7238 블록의 전체 배치는 생존에서 약 2시간, 크리에이티브에서 약 6분이 걸린다.

20.2 시간 비용 비교

T_{block} 을 블록당 배치 시간 (전형적으로 생존에서 1–2s, 크리에이티브에서 0,1–0,5s)으로, T_{clone} 을 클론당 명령 시간 (약 50ms)으로 한다. 대칭이 있는 경우와 없는 경우의 총 건설 시간은:

$$T_{\text{octant}} + 7T_{\text{clone}} \ll T_{\text{full}}$$

생존에서 $N = 7238$ 의 경우 ($T_{\text{block}} = 2\text{s}$, $T_{\text{clone}} = 0,05\text{s}$): 전체 = 14476s $\approx$ 4h 1min; 팔분원 + 클론 = 1810s + 0,35s $\approx$ 30min. 반사 부분군의 차수와 정확히 일치하는 8배 속도 향상. 차수 48의 완전한 팔면체군 O_h 는 원칙적으로 더 줄일 수 있지만 (차수 48만큼), 대각선 회전을 활용하려면 45° 회전된 빌드 영역이 필요하며, 바닐라 Minecraft에서 설정 비용을 들일 가치가 거의 없다.

21. 블록 텍스처와 시각적 구성

Minecraft 블록은 평평하게 색칠된 셀이 아니라 6개의 텍스처 면이 있는 정육면체이다. Block Round는 각 복셀을 바닐라 리소스 팩에서 샘플링된 6개의 표준 면 텍스처로 렌더링한다. 수학적으로 텍스처링 연산은 UV 매핑이다: 단위 정육면체의 각 면이 $[0, 1]^2$ 좌표로 매개변수화되고 해당 텍스처가 그 UV에서 샘플링되어 렌더링된 픽셀을 생성한다.

21.1 여섯 면과 UV 매핑

각 복셀 면에 대해 렌더러는 블록 모델을 참조한다:

$$\text{면} \in \{\text{위, 옆, 아래}\} \quad UV : [0, 1]^2 \rightarrow \text{pixels}$$

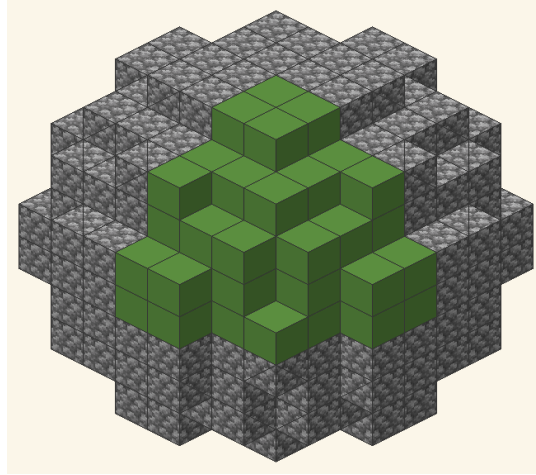
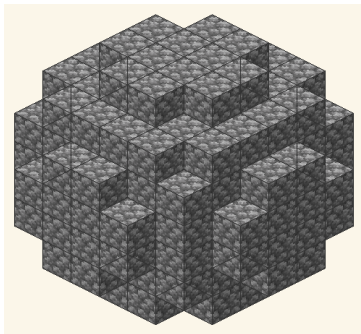
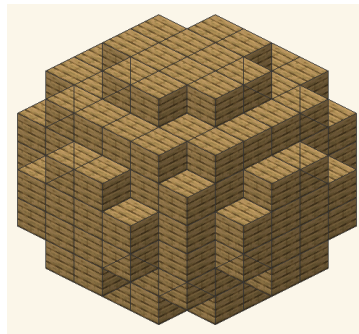


Figure 7: $D = 10$ 구에서 양의 팔분공간 $(+x, +y, +z)$ 을 초록으로 강조 표시. 부분군 $\mathbb{Z}_2^3 \leq O_h$ (위수 8, 세 좌표 반사로 생성됨) 의 작용 아래 이 팔분공간이 구 전체를 결정한다 — 나머지 일곱 영역은 수작업으로 만든 팔분공간에 *mode:masked* 옵션을 준 `/clone` 호출을 차례로 적용해 얻는다.

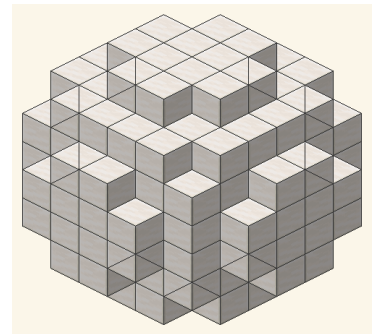
대부분의 블록 (돌, 조약돌, 다이아몬드 블록, 참나무 판자) 은 여섯 면에 같은 텍스처를 사용한다. 다만 블록 (잔디 블록, 호박, 건초 더미, 석영 기둥, 모든 나무 통나무, 작업대, 화로, 책장, TNT) 은 위, 옆, 아래에 다른 텍스처를 사용한다: 잔디 블록은 녹색 위, 잔디-흙 옆, 순수 흙 아래; 참나무 통나무는 위와 아래에 나뭇결 고리, 네 옆에 겹질이 있다. 결정적으로, 어떤 복셀을 배치할지의 수학은 사용되는 텍스처에 의존하지 않는다—같은 지름의 다이아몬드 구와 조약돌 구의 실루엣은 동일하다. 텍스처 결정은 기하학적 알고리즘이 끝난 후 적용되는 순수하게 미학적인 층이다.



(a) Cobblestone



(b) Oak Planks



(c) Quartz Block

Figure 8: 동일한 $D = 8$ 구를 세 가지 다른 블록으로 렌더링한 결과. 복셀화된 실루엣 (수학) 은 세 경우에 모두 동일하다: 알고리즘이 복셀을 선택하고, 텍스처는 이후에 적용되는 순전히 미적인 층으로서 부피도 위상도 바뀌지 않는다.

21.2 블록 가족과 톤

수십 개의 사용 가능한 블록 중에서 선택하는 생존 빌더에게 관련 질문은 텍스처 인덱스가 아니라 톤 가족이다: 밝음/어두움, 따뜻함/차가움, 균일/패턴화. 아래 표는 Block Round 빌드에서 가장 흔히 사용되는 블록을 분류한다:

블록	가족	톤	전형적인 사용
cobblestone	돌	회색	소박한 벽, 던전
stone	돌	회색	깨끗한 벽, 받침대
oak_planks	나무	베이지	바닥, 내부 마감
quartz_block	석영	흰색	사원, 현대적 돔
sandstone	모래	베이지	사막, 피라미드
deepslate	돌	어두움	지하, 강조

21.3 블록 혼합을 통한 그라데이션

단일 블록 유형은 하나의 톤을 제공하지만, 그라데이션은 층마다 두 개 또는 세 개의 블록 유형을 교차시켜 얻을 수 있다. 고전적인 Minecraft 기법은 돔의 돌에서 석영으로 그라데이션이다: 하단 층은 매끄러운 돌, 적도 층은 매끄러운 돌과 섬록암의 잡음 혼합, 상단 층은 석영. 각 층은 여전히 동일한 Block Round 알고리즘으로 계산된다; 변하는 것은 복셀당 텍스처 조회이다. Block Round의 *Random* 블록 옵션은 기본적으로 이러한 절차적 혼합을 구현한다 (잔디 캡, 그 아래 흙, 중간에 산재한 광석, 기반암 근처에 심층암). 수학은 19 절의 층 분해와 같다; 복셀당 “블록 태그”만 다르다.

22. 결론

본 문서는 Block Round에서 사용되는 수학적 기초를 체계적으로 설명했다. 약 1000 줄의 JavaScript 코드와 지원 텍스처 및 오디오 파이프라인에서, 프로젝트는 여러 이론 영역의 기여를 통합한다:

- **해석기하**: 속함의 주요 기준으로서의 타원과 타원체의 음함수 방정식.
- **고전적 래스터화 알고리즘**: 중점 공식의 Bresenham, 타원에 대한 Pitteway/Kappel 확장 포함.
- **디지털 위상**: 이산 경계 연산자, 4-, 6-, 26-연결성 개념, 슬라이스 합성 및 노출된 면 카운트.
- **적분학**: 삼중 적분으로서의 타원체 부피와 그 이산 대응물 (복셀에 대한 계수

측도).

- **선형대수**: 선형 부등식으로 정의된 절단 평면과 3 차원 원근 투영.
- **삼각법**: 구면 좌표에서 카메라 매개변수화와 FOV 에 따른 자동 거리 조정.
- **조합론적 인벤토리 산술**: 64 의 팩과 3 456 아이템의 더블 상자에 대한 올림 나눗셈 회계, 정보 칩이 표시하는 스택-나머지 표현.
- **군 대칭**: 팔면체군 O_h 와 그 차수 8 반사 부분군 $\mathbb{Z}_2^3$ 을 활용하여 `/clone` 을 통해 건설 비용을 8 배 감소.

연구가 형성할 수 있는 중심적 관찰은 다음과 같다: 이산 격자에서, 연속 곡선의 **단일한 올바른 표현은 존재하지 않는다**. 본 문서에서 제시된 각 알고리즘은 셀 포함 기준의 다른 수학적 정의에 해당하며, 각 정의는 고유한 형식적 속성을 갖는 실루엣을 생성한다—유클리드의 부드러움, Bresenham 의 계단 패턴, 모서리 적용 범위 기준의 더 큰 범위. 따라서 알고리즘 선택은 의도된 시각적 표현, 결과 형상의 원하는 측정 특성, 그리고—Block Round 에 특히—게임에서 형상을 물리적으로 건설하는 인벤토리 및 시간 비용을 고려해야 하는 기술적 결정이다.

Block Round 는 Minecraft 도구 생태계에서 작지만 수학적으로 풍부한 틈새를 차지한다: 순수 기하 도구 (WorldEdit, Litematica) 와 순수 시각 도구 (리소스 팩, 셰이더) 사이에 위치하여, 연속 방정식에서 이산 배치 계획에 이르는 명시적이고, 도출 가능하며, 재현 가능한 다리를 제공한다. 자매 프로젝트 Pixel Round 는 Minecraft 층 없이 동일한 알고리즘을 제공하여 전통적인 픽셀 아트와 복셀 아트 응용에 적합하다. 두 프로젝트는 함께 동일한 연속에서 이산으로의 수학적 파이프라인이 어떻게 근본적으로 다른 예술적 및 건설적 워크플로를 지원하는지 예시한다.

참고문헌

기술 참고문헌 (래스터화 및 디지털 위상수학)

- **Bresenham, J. E.** *Algorithm for computer control of a digital plotter*. IBM Systems Journal, vol. 4, n. 1, p. 25–30, 1965. 이용 가능한 곳 [doi:10.1147/sj.41.0025](https://doi.org/10.1147/sj.41.0025). — 본 문서의 §5 에서 다루는 정수 중점 알고리즘의 원논문.
- **Pitteway, M. L. V.** *Algorithm for drawing ellipses or hyperbolae with a digital plotter*. The Computer Journal, vol. 10, n. 3, p. 282–289, 1967. 이용 가능한 곳 [doi:10.1093/comjnl/10.3.282](https://doi.org/10.1093/comjnl/10.3.282). — Bresenham 알고리즘을 임의의 원뿔곡선으로 일반화한 것으로, §5 에서 설명한 타원 확장의 기초.
- **Kappel, A.** *An ellipse-drawing algorithm for raster displays*. 수록 *Fundamental*

Algorithms for Computer Graphics (R. A. Earnshaw, ed.), NATO ASI Series F-17, Springer, 1985, p. 257–280. — 타원 구현에서 사용한 두 영역 정식화.

- Klette, R.; Rosenfeld, A. *Digital Geometry: Geometric Methods for Digital Picture Analysis*. Morgan Kaufmann, 2004. 656 쪽. — 디지털 위상수학, 경계 작용소 (∂F), 4-/6-/26-연결성의 표준 참고문헌. §7, §12, §18 에서 사용.
- Foley, J. D.; van Dam, A.; Feiner, S. K.; Hughes, J. F. *Computer Graphics: Principles and Practice*. 제 3 판, Addison-Wesley, 2014. — 래스터화, 조명 모델, 원근 투영의 고전적 논의. §13–15 와 관련.

참조한 Minecraft 사양 및 도구

- Sponge Project. *Sponge Schematic Specification, version 2*. github.com/SpongePowered/Schematic-Specification. — Block Round 가 보내내는 .schem 파일 형식 (gzip + NBT). WorldEdit, Litematica, MCEdit 으로 불러올 수 있다.
- EngineHub. *WorldEdit Documentation*. worldedit.enginehub.org. — 본 문서 전반에서 인용되는 `//sphere`, `//hsphere`, `//ellipsoid`, `//cyl`, `//schem load` 명령에 대한 공식 참고문헌.
- Mojang AB. *Minecraft Wiki — Inventory*. minecraft.wiki/w/Inventory. — §17 에서 사용되는 슬롯, 스택, 단일 상자 (27 슬롯) 및 더블 상자 (54 슬롯)에 대한 참고자료.
- Mojang AB. *Minecraft Wiki — Commands/fill, /clone*. minecraft.wiki/w/Commands/fill, minecraft.wiki/w/Commands/clone. — §17 과 §20 에서 인용되는 바닐라 명령 `/fill` 과 `/clone`.
- Masady (*Litematica* mod). github.com/maruohon/litematica. — 스키매틱을 불러와 시공할 건축물의 반투명 고스트를 표시하는 Minecraft Java Edition 모드.

기하학, 선형대수, 군론

- 김홍중 선형대수와 그 응용. 경문사, 2012. — 타원의 음함수 방정식, $\mathbb{R}^3$ 에서의 평면 대수, 구면 좌표. §3, §9, §13 내용의 기초.
- Armstrong, M. A. *Groups and Symmetry*. Undergraduate Texts in Mathematics, Springer, 1988. — 팔면체군 O_h 와 그 부분군에 대한 입문적 서술, §20 에서 사용.
- Coxeter, H. S. M. *Regular Polytopes*. 제 3 판, Dover, 1973. — 정다면체 대칭

군의 표준 참고문헌으로, §20 에서 언급된 위수 48 의 O_h 에 대한 자세한 처리를 포함.

- Hilbert, D.; Cohn-Vossen, S. *Geometry and the Imagination*. AMS Chelsea, 1990 [orig. 1932]. — 이차곡면과 기하학적 시각화의 고전적 논의, §9 의 직관적 기초.

알고리즘, 자료구조, 복잡도

- Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. *Introduction to Algorithms*. 제 4 판, MIT Press, 2022. — §4–6 의 세 알고리즘에 대한 복잡도 분석, §17 의 천장 나눗셈.
- Wirth, N. *Algorithms + Data Structures = Programs*. Prentice-Hall, 1976. — Bresenham 의 직선 알고리즘 및 그 원주로의 확장에 관한 고전적 서술.

보조 온라인 자료

- Block Round — 앱: vinisouza128.github.io/block-round/.
- Block Round — 저장소: github.com/ViniSouza128/block-round.
- 자매 프로젝트 Pixel Round(Minecraft 텍스처 없는 버전): github.com/ViniSouza128/pixel-round.
- pt-BR 수업 계획서 (고등학교 3 학년): docs_aula/Plano_de_Aula_pt-BR.pdf.

부록 A: 건설 참조표와 실제 예제

이 부록은 Block Round 사용자가 가장 많이 사용하는 수치 및 명령 참조를 모은다. 아래 데이터는 문서 곳곳에 흩어진 절별 표와 실제 예제를 통합하여, 건설 계획, 명령 구문, 그리고 종단 간 실제 예제를 위한 단일 페이지 조회로 기능하도록 구성된다.

A.1 종합 구 참조 (채움, 비움, 팩)

아래 표는 10 절과 17 절의 표준 구 표를 채운 부피와 함께 비어 있는 셀 변형 (얇은 모드) 을 추가하여 확장한다. 비어 있는 셀 카운트는 단일 복셀 셀을 가정하고 유클리드 알고리즘으로 계산된다; 두꺼운 모드는 12 절에 설명된 대로 대각선 틈을 막기 위해 약 10–15% 더 많은 블록을 추가한다.

D	채움 (V)	비움 (셀)	팩 채움	팩 비움
8	268	170	5	3
10	523	316	9	5
12	904	500	15	8
14	1437	744	23	12
16	2145	1042	34	17
18	3052	1338	48	21
20	4189	1648	66	26
24	7238	2400	114	38
28	11494	3296	180	52
32	17156	4332	269	68

생존 모드 빌더에게 비움 열이 일반적으로 관련된다: 보이는 셀만 전용 텍스처 블록이 필요하고, 내부 (원한다면) 는 가장 저렴한 재료로 채울 수 있다. 팩-채움 열은 완전히 채운 구의 총 비축 크기를 알려준다; 팩-비움은 비어 있는 돔이나 셀의 현실적 최소값이다.

A.2 WorldEdit / 바닐라 명령 참조

다음 표는 게임 내에서 Block Round 출력을 구성하는 데 가장 관련 있는 WorldEdit 및 바닐라 Minecraft 명령을 요약한다. WorldEdit 명령은 플레이어가 마법봉 (기본 나무 도끼) 을 들고 영역을 선택했다고 가정한다; 바닐라 명령은 모드 없이 작동하지만 좌표 인수가 필요하다.

명령	모드 / 출처	결과
//sphere stone 8	WorldEdit	채워진 구 $r=8$ (유클리드)
//hsphere stone 8	WorldEdit	비어 있는 구 $r=8$ (셀만)
//cyl stone 8 16	WorldEdit	원기둥 $r=8, h=16$
//ellipsoid st. 10 5 10	WorldEdit	타원체 $10 \times 5 \times 10$
/fill ... stone	바닐라	한 블록 종류로 박스 채우기
/clone ... mode:masked	바닐라	영역을 새 위치로 복사

Litematica 사용자의 경우 워크플로가 다르다: 명령을 실행하는 대신, schematic 파일 (Block Round 은 .schem, Sponge 형식 v2 를 내보낸다) 을 Litematica 모드에 로드하면, 모드는 플레이어가 블록 단위로 배치하는 도형의 반투명 유령을 표시한다. 이것은 WorldEdit 의 크리에이티브 경험에 가장 가까운 생존 모드 등가물이다.

A.3 실제 예제: 신전 위의 $D=20$ 석영 돔

완전한 실제 예제로, 빌더가 기존 돌 신전 위에 $D=20$ 의 석영 돔을 원한다고 가정한다. 신전 꼭대기는 11×11 블록이고, 세계 좌표 $(0, 80, 0)$ 에 중심이 있다. 돔은 $Y=0$ 평면에서 잘린 반지름 $r=10$ 구의 상반부이다. 건설 계획:

- Block Round 로 계획.** 앱을 열고 3D 모드, 구, 크기 $D=20$, 절단 축 Y 를 50% 로 설정한다. 피커에서 석영 블록을 선택한다. 정보 칩은 $V_{\text{dome}} \approx 2126$ 블록 (약 34 팩, 두 번째에 1330 아이템이 남는 더블 상자 1 개) 을 보고한다.
- 자원 모으기.** 34 팩의 석영 블록 = $\lceil 2126/4 \rceil = 532$ 석영 블록 $\times 4$ 네더 석영 각 = 2128 네더 석영. 적절히 제련하거나 거래한다.
- 알고리즘 결정.** 아래에서 보는 석영 돔의 경우 유클리드 알고리즘이 가장 부드러운 실루엣을 제공한다; 외딴 산의 돔이라면 임계값 알고리즘이 더 깔끔하게 보인다. Block Round 는 기본적으로 유클리드를 사용한다.
- 돔 배치.** 돔의 평면은 신전 꼭대기와 일치해야 한다. 돔의 경계 상자는 $20 \times 10 \times 20$ 블록이다; 중심을 $(0, 80, 0)$ 에 배치하면 평면 (절단) 면이 $y=80$ 에 놓이고 돔 정점이 $y=89$ 에 놓인다.
- 대칭으로 건설.** 돔의 $+x, +z$ 사분면 (약 530 블록) 을 건설한다. `/clone 0 80 0 10 89 10 -10 80 0 mode:masked` 를 발행하여 x 에서 반사; 그 다음 `/clone -10 80 0 10 89 10 0 80 -10 mode:masked` 를 z 에서 반사. 총 경과 시간: 생존 모드에서 약 10 분.
- 마무리.** 정점과 적도의 네 기본 방위점에 바다 랜턴을 추가하여 발광 조명을 한다 (Block Round 는 이를 MC 광 레벨 15 의 발광 맵으로 렌더링한다). 마치

막 몇 블록을 배치하기 전에 돔 셀에 간격이 없는지 확인하기 위해 선택적으로 Block Round 의 가장자리 오버레이 (키 `g`) 를 토글한다.

각 단계의 수학은 정확히 앞의 22 개 절이 설명하는 것이다: 음함수 타원체 방정식, 유클리드 복셀화, Y 축 절단, `/clone` 을 통한 팔면체 대칭 감소, 부피를 팩과 상자로 번역하는 인벤토리 산술. 따라서 부록은 새로운 이론이 아니다—이 문서의 모든 이론적 개념이 건설 계획 워크플로의 단일하고 구체적인 결정에 매핑된다는 사실을 상기시키는 것이다.

A.4 키보드 단축키 참조

Block Round 앱은 캔버스의 모서리 버튼을 반영하는 작은 키보드 단축키 모음을 노출한다. 빠른 참조를 위해 아래에 나열되어 있다; 단축키는 전역적이며 (입력 필드에 포커스가 필요하지 않음), 별도 표시가 없는 한 2D 와 3D 모드 모두에서 작동한다.

키	전환	참고
G	가장자리 그리드 오버레이	각 복셀 면의 검은 윤곽
C	중심 가이드	축에 반투명 노란색 십자
D	PNG 다운로드	현재 캔버스를 PNG 이미지로
I	정보 칩	블록 수와 팩 분류
M	2D / 3D 전환	평면과 복셀 모드 전환
S	소리 켜기/끄기	주변 블록 배치 소리
T	낮 / 밤 테마	배경을 어둡게 하고 타일을 읽기 쉽게 유지
Ctrl+Z / Y	실행 취소 / 다시 실행	도형 변경 기록을 따라감

단축키 `g`(그리드) 와 `c`(중심 가이드) 는 건설 검증 중에 특히 유용하다: `g` 는 셀의 모든 복셀이 제자리에 있는지 보여주고 (누락된 블록은 오버레이 패턴을 깬다), `c` 는 절단이 도형 축에 적절히 중심에 있는지 확인한다.

Ctrl+Z 실행 취소의 경우: Block Round 는 도형을 변경하는 모든 편집 (슬라이더, 렌더링 모드, 알고리즘, 형태, 축, 블록, 모드 전환) 을 따라가지만 의도적으로 시각적 전환만 (카메라, 가장자리 오버레이, 중심 십자, 테마, 소리) 은 제외한다. 이 분리는 사용자가 되돌리고 싶어할 편집들로 실행 취소 스택을 채운다.

A.5 핵심 용어 용어집

이 문서 전체에 반복적으로 나타나는 기술 용어에 대한 빠른 참조. 각 항목은 개념을 간단히 요약하고 상세히 전개되는 절을 가리킨다.

- **복셀** — 3D 의 단위 정육면체, 2D 픽셀의 유사물. 각 Minecraft 블록은 하나의 복셀이다. 제 2 절 참조.
- **팩 (스택)** — Minecraft 에서, 인벤토리 한 칸을 차지하는 64 개의 동일한 아이템의 스택. 제 17 절 참조.
- **더블 상자** — 두 인접 상자가 54 칸 컨테이너로 병합된 것. 최대 3456 개의 아이템을 보유한다. 제 17 절 참조.
- **셜커 상자** — 상자 칸 내부에 저장 가능한 27 칸 휴대용 컨테이너. 제 17 절 참조.
- **셀** — 단단한 도형의 외부 한 복셀 두께 층. 생존 빌더와 관련된 표면. 제 7 절과 제 18 절 참조.
- **팔분원** — 세 좌표 반공간의 교차로 얻어진 3D 공간의 여덟 영역 중 하나. 대칭 최적화에 사용된다. 제 20 절 참조.
- **UV 매핑** — 복셀의 각 면에 텍스처 이미지를 매핑하는 데 사용되는 3D 면의 2D 매개변수화. 제 21 절 참조.
- **Bresenham** — 정수 산술만 사용하는 1965 년의 증분 래스터화 알고리즘. 제 5 절 참조.
- **임계값** — 셀의 어떤 모서리가 타원 내부에 있으면 셀을 포함하는 래스터화 기준. 제 6 절 참조.
- **유클리드** — 셀의 중심이 타원 내부에 있으면 셀을 포함하는 래스터화 기준. 제 4 절 참조.
- **이스터에그** — 특정 입력으로 발동되는 숨겨진 기능. Block Round 에서: 참나무와 크리퍼, 둘 다 슬라이더 값 15 에서.
- **스키매틱** — WorldEdit, Litematica 또는 MCEdit 으로 로드 가능한 직렬화된 구조 파일 (Sponge Schematic v2, .schem).

이 용어집은 의도적으로 완전한 수학 사전보다 짧게 유지된다. 기초 수학 (디지털 위상, 적분학, 군론 등) 의 더 깊은 다름을 원하는 독자는 이 문서의 관련 절을 참조하기 바란다.

A.6 모델의 전망과 한계

이 문서에서 제시된 Block Round 모델은 타원, 타원체 및 그 절단의 정적 복셀화와 Minecraft 층이 도입한 실용적 고려사항을 다룬다. 동적 문제—움직이는 도형, 애니메이션 복셀 전환, 유체 시뮬레이션과의 상호작용—은 이산 래스터화 범위

밖의 연속 시간 프레임워크를 요구하기 때문에 다루지 않는다. 자연스러운 확장 은 스키매틱 수준의 애니메이션이다: 보간으로 연결된 정적 복셀화의 시퀀스로, Block Round 는 이미 Litematica 스크립트에서 읽을 수 있는 .schem 스트림으로 내보낼 수 있다.

또 다른 열린 방향은 정확한 부피 보존 복셀화이다: 현재의 이산 카운트 V_{disc} 는 연속 부피 V_{cont} 에 점근적으로만 수렴한다. 정확한 카운트가 연속 공식과 일치해야 하는 응용 (예: 렌더링 대회, 엄격한 블록 예산의 수학적 예술) 의 경우 추가 정제 층이 필요하다—예를 들어, 이산 카운트가 목표 값에 도달하도록 반지름을 약간 조정. Block Round 는 현재 이 정제를 구현하지 않지만, 기본 수학은 간단하다: $V_{\text{disc}}(r) = V_{\text{target}}$ 을 r 에 대해 수치적으로 푸는 것.

마지막으로 고차 대칭 활용: 본 문서는 O_h 의 차수 8 반사 부분군을 다루지만, 차수 48 의 완전한 군은 원칙적으로 $\times 48$ 속도 향상을 허용하는 회전을 포함한다. 실질적 장애물은 게임 내 명령이 축 정렬 영역에서 작동하고, 회전 대칭을 활용하려면 45° 회전된 빌드 영역이 필요하다는 것이다. 선택 상자를 내부적으로 회전하는 서버 모드 (예: FAWE) 는 원칙적으로 이를 달성할 수 있지만, 주류 도구는 아직 공개하지 않는다. 바닐라 서버에서 $\times 48$ 속도 향상을 시연하는 참조 구현은 본 연구의 자연스러운 후속이 될 것이다.

A.7 추가 참조 및 외부 리소스

이 문서에서 전개된 수학적 개념들은 각각 잘 확립된 문헌을 가지고 있다. 원본 자료나 더 깊은 알고리즘 배경을 찾는 독자에게는 다음 진입점을 권장한다:

- J. E. Bresenham, *Algorithm for computer control of a digital plotter*, IBM Systems Journal, vol. 4 no. 1, 1965 이 문서에서 확장된 정수 산술 래스터화의 원본 논문.
- M. L. V. Pitteway, *Algorithm for drawing ellipses or hyperbolae with a digital plotter*, Computer Journal, vol. 10, 1967 Bresenham 을 임의의 원뿔 곡선으로 일반화.
- A. Kappel, *An ellipse-drawing algorithm for raster displays*, ACM Comm. vol. 28, 1985 5 절에서 사용된 두 영역 타원 공식.
- R. Klette 과 A. Rosenfeld, *Digital Geometry: Geometric Methods for Digital Picture Analysis*, Morgan Kaufmann, 2004 7 절과 18 절의 디지털 위상 및 경계 연산자에 대한 표준 참고.
- WorldEdit 문서, enginehub.org/worldedit/ 본 문서 전체에서 사용된 `//sphere`, `//hsphere`, `//ellipsoid`, `//copy` 명령군의 공식 참조.
- Sponge Schematic 사양 v2, github.com/SpongePowered/Schematic-Specifica

tion Block Round 가 .schem 으로 내보내는 파일 형식.

22 절에 나열된 여덟 개의 이론 영역은 각각 단일 기술 문서가 요약할 수 있는 범위를 훨씬 넘는 교과서 수준의 처리를 가지고 있다. Block Round 의 기여는 이론 자체에 있지 않고 특정한 종합에 있다: 일반적으로 순수 기하 도구에서 생략되는 명시적 인벤토리 및 대칭 비용 고려와 함께 텍스처화된 복셀 Minecraft 환경에 고전적 래스터화를 적용하는 것.

재현성에 대한 마지막 발언: 본 문서의 전체 파이프라인 빌드 스크립트, 번역, LaTeX 템플릿, 로케일별 테이블 은 프로젝트 저장소의 docs_math/ 디렉토리에 포함되어 있다. XeLaTeX/MiKTeX 이 있는 시스템에서 python build.py 를 다시 실행하면, 지원되는 아홉 개 로케일 중 어느 것에서나 이 PDF 를 정확히 생성한다. 자매 프로젝트 Pixel Round 는 동일한 구조를 따라 텍스처화 및 비텍스처화 변형의 직접적인 나란히 비교를 허용한다.

문서 끝

Block Round 는 Vinícius Rodrigues de Souza 가 유지 관리한다. 자매 프로젝트 Pixel Round 는 github.com/ViniSouza128/pixel-round에서 이용 가능하며; 현재 Block Round 저장소는 github.com/ViniSouza128/block-round에 있다.

의견, 수정 및 번역 개선은 프로젝트 저장소의 이슈 트래커를 통해 환영한다. 영어가 아닌 여덟 개 로케일의 원어민 교정은 특히 중요하게 평가된다. 현지화는 상당한 검토를 거쳐 준비되었지만 기술적 - 수학적 용어는 원어민의 교정으로부터 혜택을 받는다.